**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# LESSON 130 · Programming: Files, SQL and Strings

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. What is a parameter?

Q2. Which is a benefit of a high-level language?
A. Never needs translation
B. Easier for humans to read
C. Runs directly on every CPU
D. Uses no programming rules

Q3. “Adding active users always increases the bandwidth available to each user.” Correct this. Explain why.

Q4. A school needs central accounts and shared files. Three club laptops share files directly. Correct both claims. Explain why. (a) Every client must be a dedicated server. (b) Peer-to-peer needs a central server.

**Do Now** Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# I Do · Read new knowledge

### Key vocabulary

String handling — Concatenate and extract parts of text when cleaning/formatting data.

File processing — Open, read/write and close files in the appropriate mode.

SQL retrieval — SELECT selects fields, FROM selects table and WHERE filters rows.

Combined tasks — A realistic system may read records, filter content and produce a formatted result.

### New learning

Link the representations: The same three pupils can appear as CSV text lines, a 2D table and a typed SQL database. The representation changes, not the facts.

Separate indexing from filtering: A 2D array uses indexes for cell access; a loop and IF are needed to select every row satisfying a condition.

SQL query: SELECT chooses displayed fields, FROM names the table and WHERE selects matching records.

Optional integration: Loading CSV rows into an array and converting string marks into numbers is an ambitious extension; start with a supplied array for core work.

### Worked example

Compare SQL and code for the shared Pupils table. Show why 'SELECT Name FROM Pupils WHERE Mark >= 50' returns Ada and Mia.

1. Read marks 72, 43 and 81 from the Pupils table; identify that 50 is the minimum included mark.

2. SQL SELECT Name requests only the Name field; WHERE Mark >= 50 keeps Ada and Mia.

3. In code, loop through aligned names=['Ada','Ben','Mia'] and numeric marks=[72,43,81]; test each mark>=50.

4. Print names[0] and names[2]. The code and SQL give the same set of names, although ordering is not guaranteed by SQL without ORDER BY.

# We Do · Practise with guidance

### Guided 1

Using the SAME dataset, write SQL and describe code that returns the Name of pupils with Mark < 60. [5 marks]





**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

**2. Write a complete new answer in GREEN PEN. Show working if needed:**

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

## Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. A named input received by a subprogram when called with an argument.

Q2. B — Easier for humans to read

Q3. Users may compete for finite bandwidth, reducing the data rate available to each.

Q4. (a) A server manages shared services and resources for clients. (b) Peers share directly and can act as clients and servers. Central control suits a school; peer-to-peer can cost less for a small club but is harder to manage as it grows.

### I Do / We Do pages · Guided practice

Q1. Equivalent code loops through indexes 0–2 and prints the name only when marks[i] < 60; output Ben.

### You Do pages · Code First

```
SELECT Title FROM Books WHERE Available = 1;
```

Expected result: titles of rows with Available = 1 (table supplied)

### You Do pages · Independent questions

Q1. SELECT Name [1] FROM Pupils [1] WHERE Mark >= 70 [1]; Ada and Mia match.

Q2. IF marks[i] < 50 THEN [2 for correct condition/control] print(names[i]) [1]; prints Ben.

Q3. readLine() supplies text [1]; '72' is a string, while 50 is numeric [1]; the field must be converted/validated before a numerical comparison [1].

### Exit Ticket · Model answer

False

## Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**



RHS COMPUTING / OCR J277 / 2.2.3 LESSON 130 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Worked method: Equivalent code loops through indexes 0–2 and prints the name only when marks[i] < 60; output Ben.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Code First · Predict → Write → Test → Improve

Write a SQL query returning all titles from Books where Available = 1.

```
SELECT Title FROM Books ...
```

write

### Question 1

Write SQL to display Name from Pupils for pupils whose Mark >= 70. [3 marks]

### Question 2

Complete a code condition so that names[i] is printed only when marks[i] < 50. State the result using the shared data. [3 marks]

### Question 3

Explain why comparing '72' directly with numeric 50 can be problematic when reading marks from a CSV text file. [3 marks]

## Exit Ticket · Answer before checking

SQL SELECT identifies the table to search.



RHS COMPUTING / OCR J277 / 2.2.3 LESSON 130 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.